Hochschule Kaiserslautern
University of Applied Sciences

Angewandte Ingenieurwissenschaften Kaiserslautern

Course of studies:

Master of Mechanical engineering / Mechatronics

Project Title:

System level rapid development in mechatronics

Submitted by:
Konda Madana Gopala Reddy
Matriculation No.: 889670

Mentor:
Prof. Dr.-Ing. Gerd Bitsch

Table of Contents

LIST OF FIGURES

5.Simulink State flow basics

5.1Simple On/Off State Chart

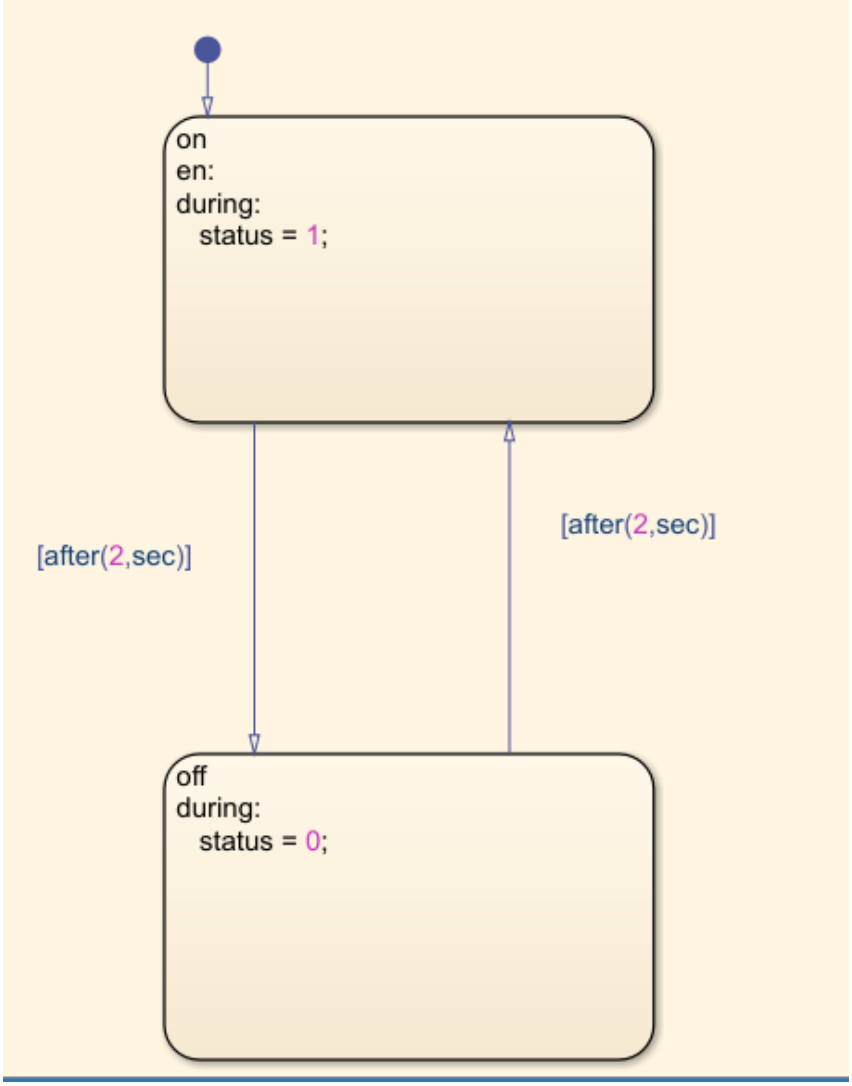

Figure 1.Simple Toggle State Machine

This state machine switches between two states: on and off. In the on state, the chart sets status = 1, and in the off state it sets status = 0. Both transitions are controlled using the condition after (2, sec), so the chart stays in each state for exactly two seconds before switching.

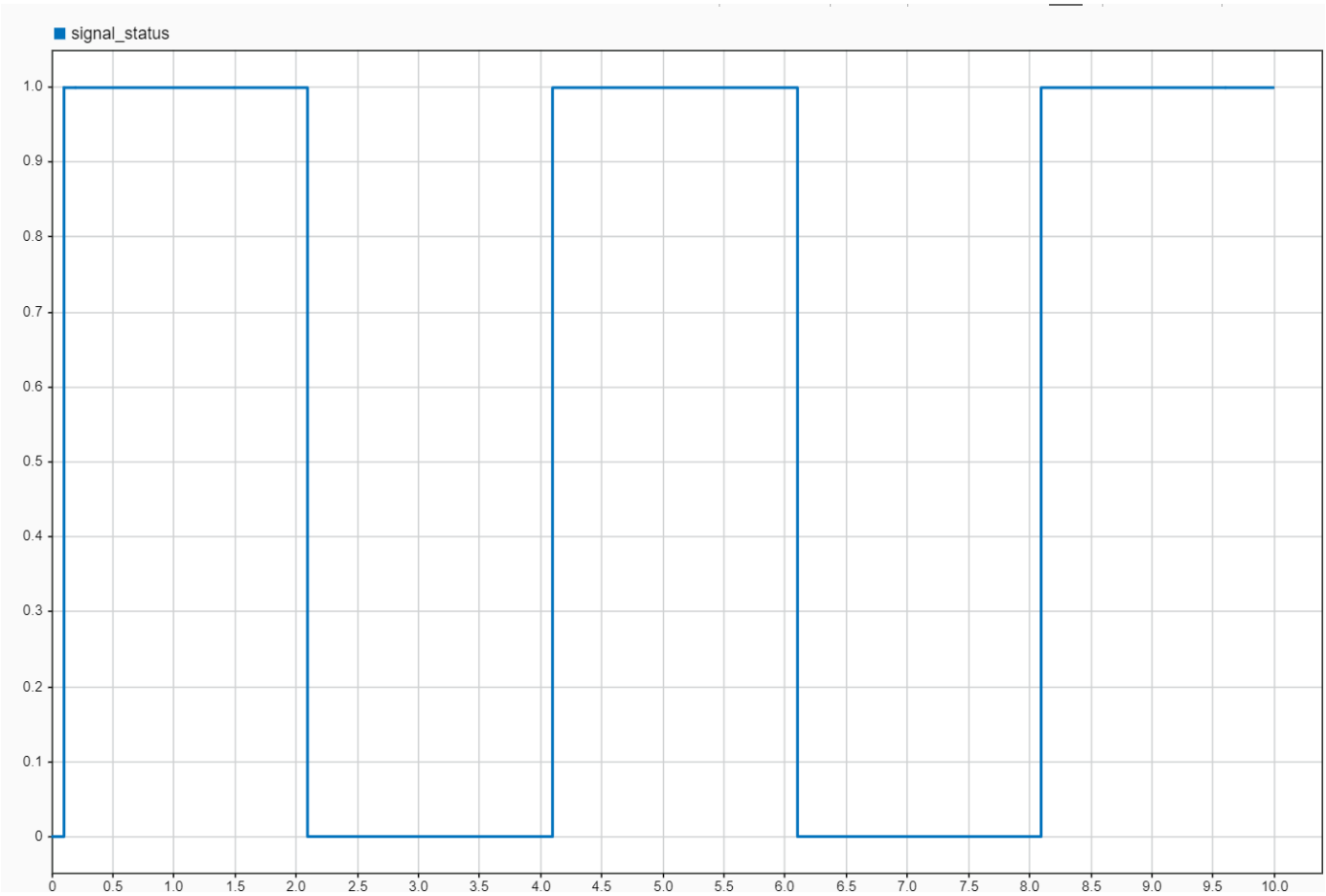

Figure 2. Simulation Output of the Status Signal

Resulting square-wave output showing the status variable toggling between 1 and 0 every 2 seconds, matching the designed state-machine behaviour.

5.2 State chart with input value

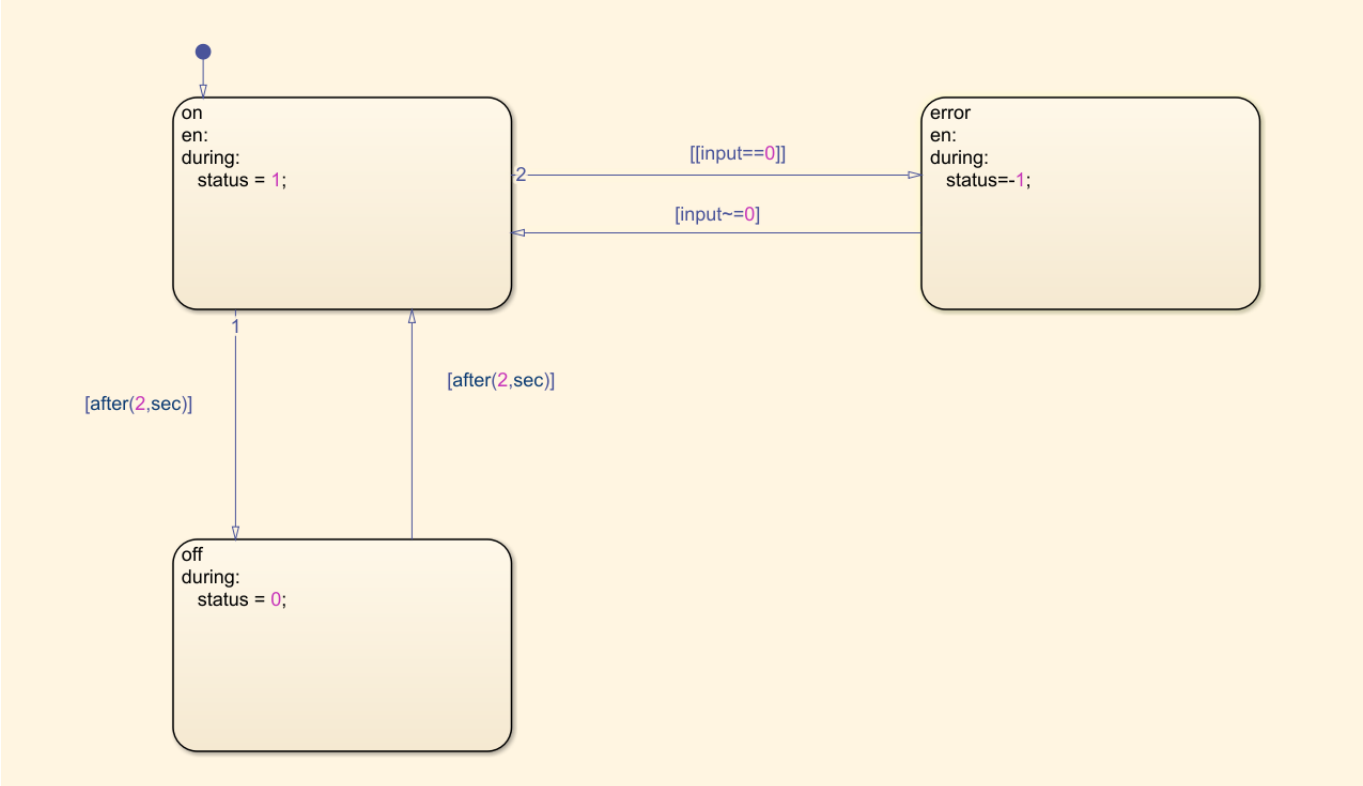

Figure3.State Machine with Input-Driven Error Transition

This extended state chart adds an input signal and an error state to the basic on/off toggle. The chart still switches between the on state (status = 1) and the off state (status = 0) every two seconds using the after (2, sec) transitions.

A new transition path has been added from both states:

- Whenever the input value becomes 0, the chart immediately leaves the normal cycle and moves into the error state.
- In the error state, status is set to –1, clearly indicating that fault condition has occurred

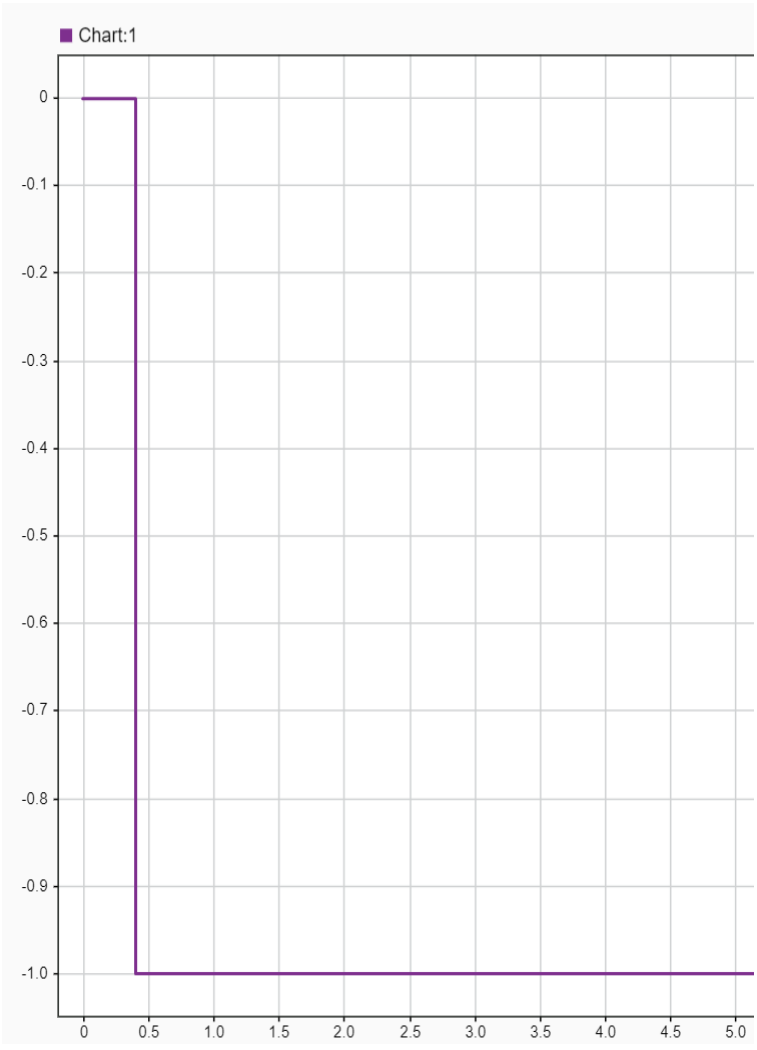

Figure 5 Simulation Output with input 0

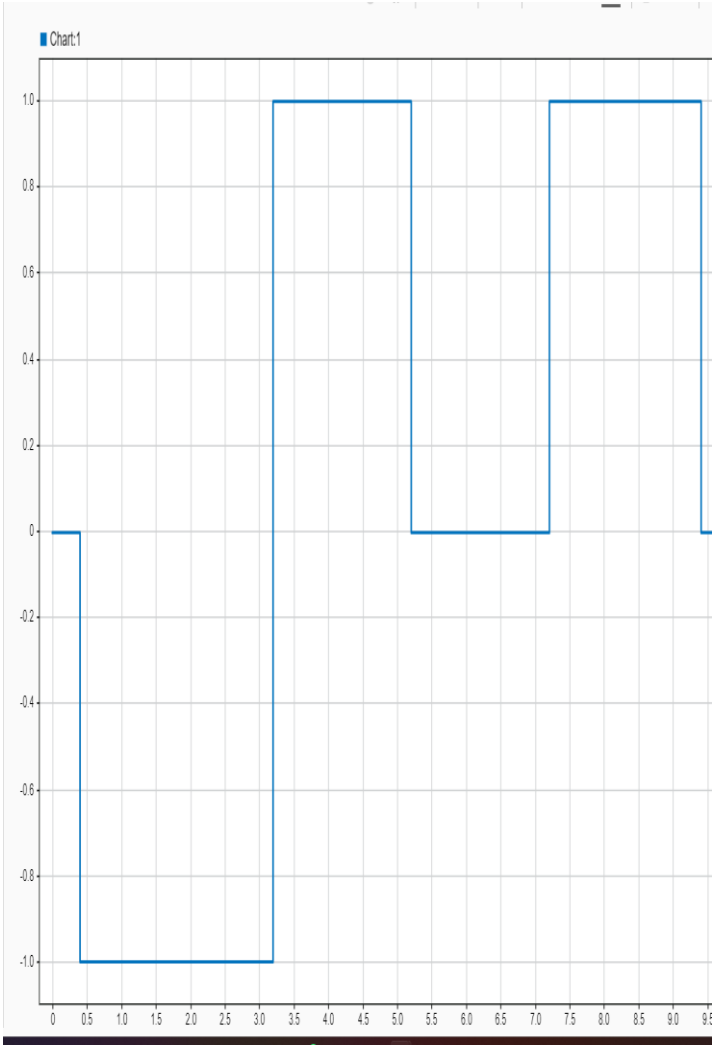

Figure 4.Simulation Output with input step

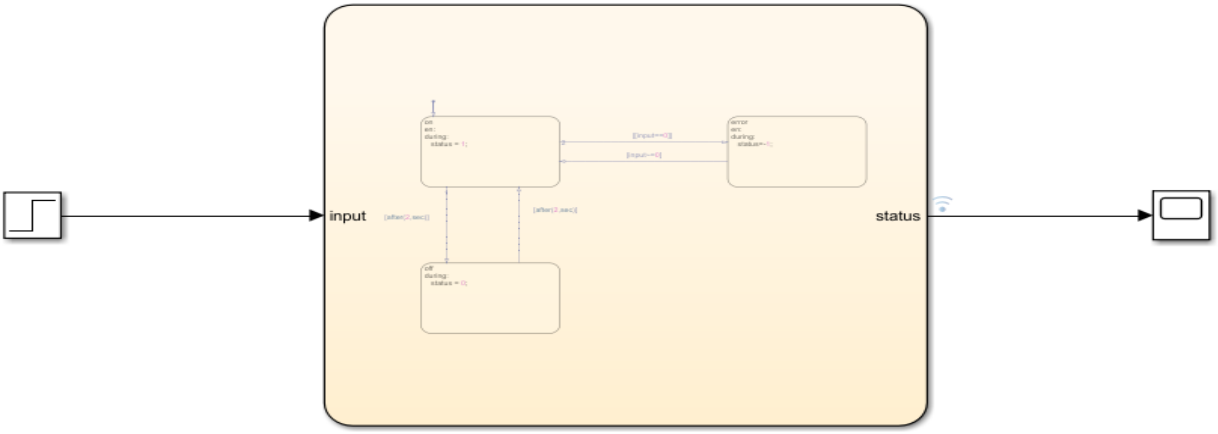

Figure 6. Simulink Model with Input Feeding the State Machine

With the input fixed at zero, the state machine bypasses the normal on/off cycle and directly enters the error state. The simulation plot clearly shows the status signal falling to –1 within the first sample and staying there, demonstrating correct error-state behaviour.

5.3 Truth Table Behaviour

Condition Table

	DESCRIPTION	CONDITION	D1	D2	D3	D4
1	condition 1	my_tt_in > 0.5	T	T	F	F
2	condition 2	my_tt_in<=0.5	F	T	F	T
		ACTIONS: SPECIFY A ROW FROM THE ACTION TABLE	1	1	1	2

Action Table

	DESCRIPTION	ACTION
1	Example action 1 called from D1 & D2 column in condition table	my_tt_out_flag = true;
2	Example action 2 called from D3 column in condition table	my_tt_out_flag = false;

Figure 7. Truth Table for Generating a Boolean Flag

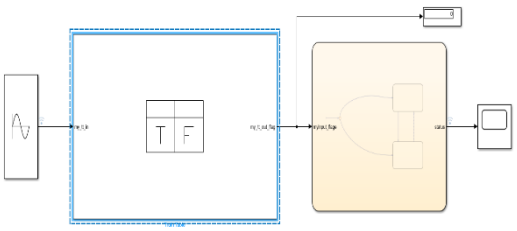

Figure 8. Truth Table Block Connected to State Machine

In this setup, the sine wave provides a continuously varying input to the truth table. The truth table evaluates this input and generates a Boolean-style flag indicating whether the signal is above the threshold. This flag is used by the State flow machine to select which state becomes active and what output value should be produced.

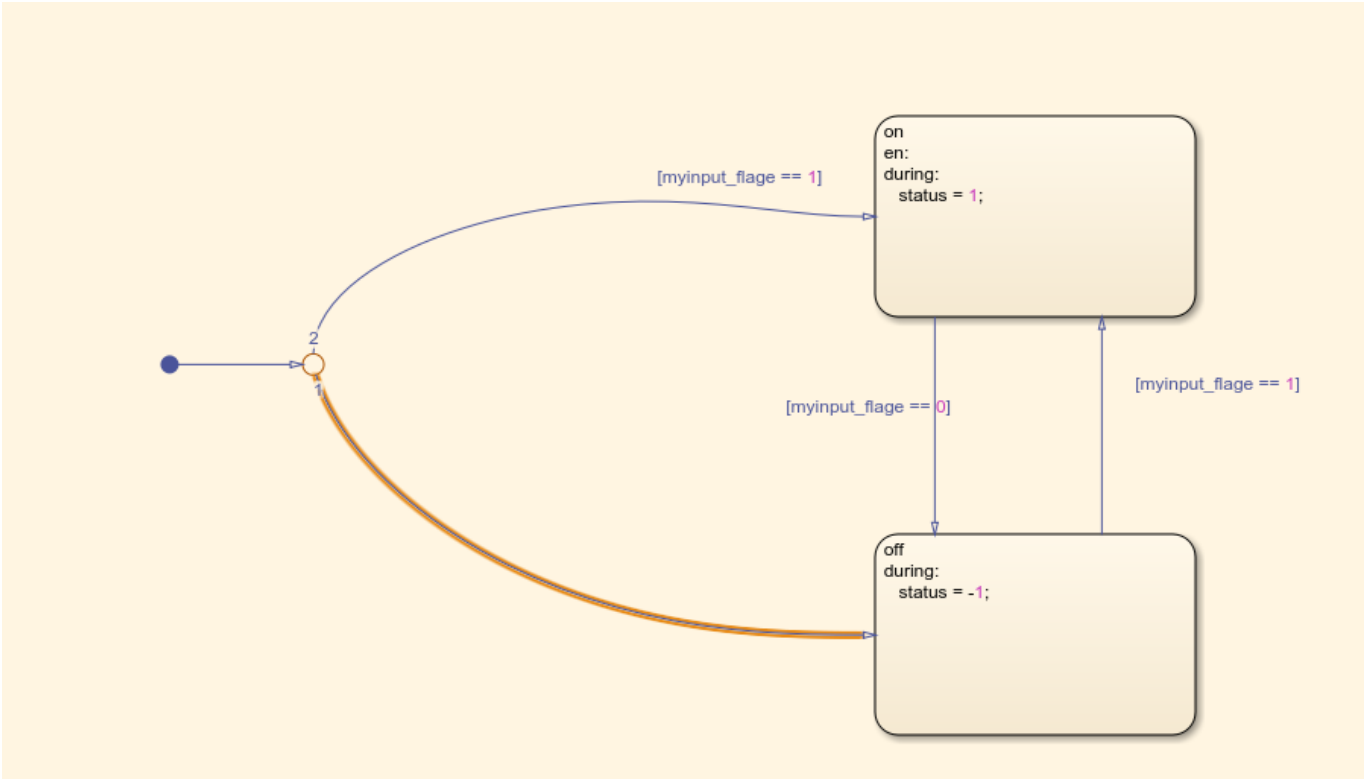

Figure 9.State Logic Based on Truth Table Output

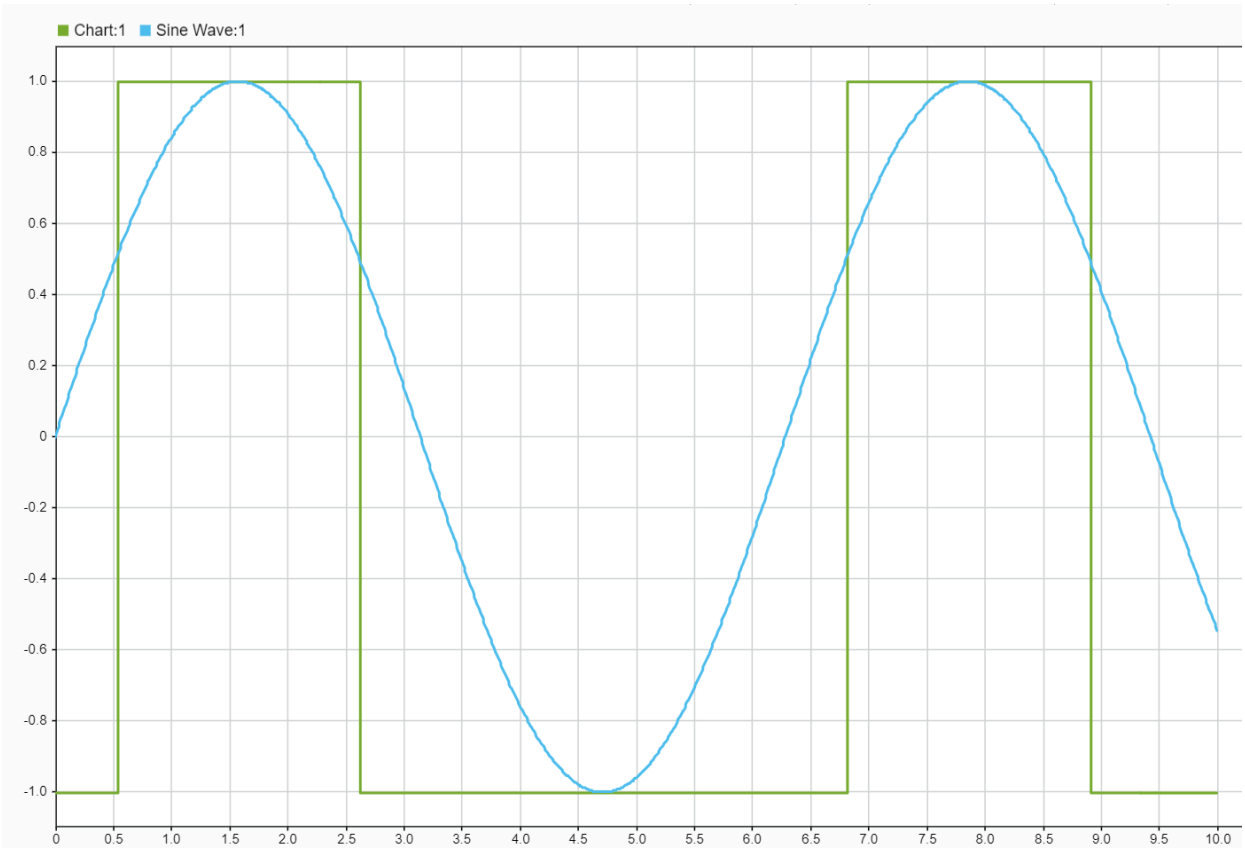

Figure 10. Simulation of Sine Input and Truth-Table Output

In this section, the sine-wave input is evaluated by a truth table to produce a flag. The state machine uses this flag to select between an “on” and “off” state, generating a clean square-wave output.

5.4 Creating a Superstate

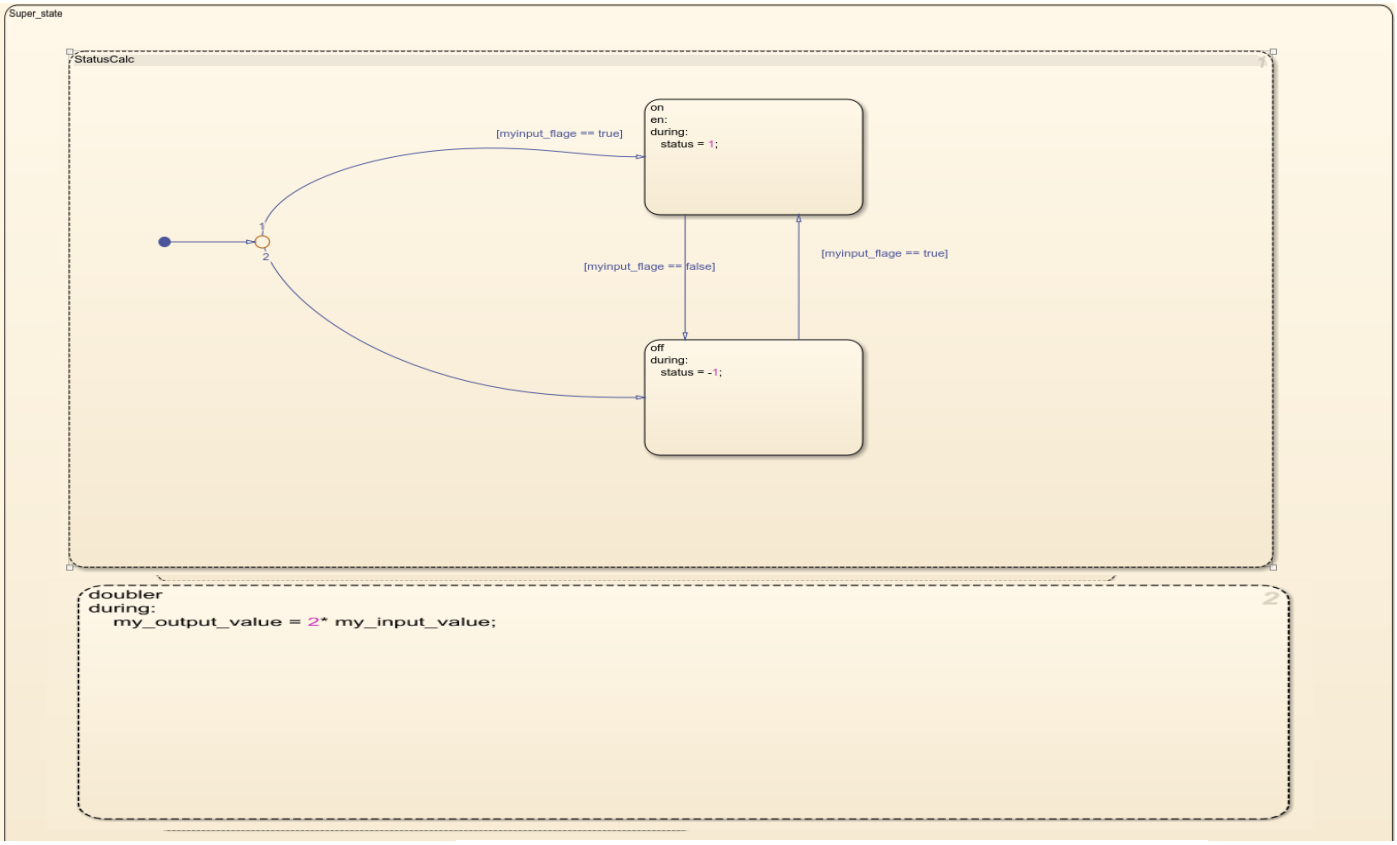

Figure 11.Superstate with Parallel States

This chart uses parallel (AND) decomposition, meaning both inner states execute simultaneously.

- In the StatusCalc region, the state machine switches between on and off depending on whether the flag is true or false, setting status to 1 or –1.
- In the doubler region, the chart calculates a second output by applying my_output_value = 2 * my_input_value; during every step.

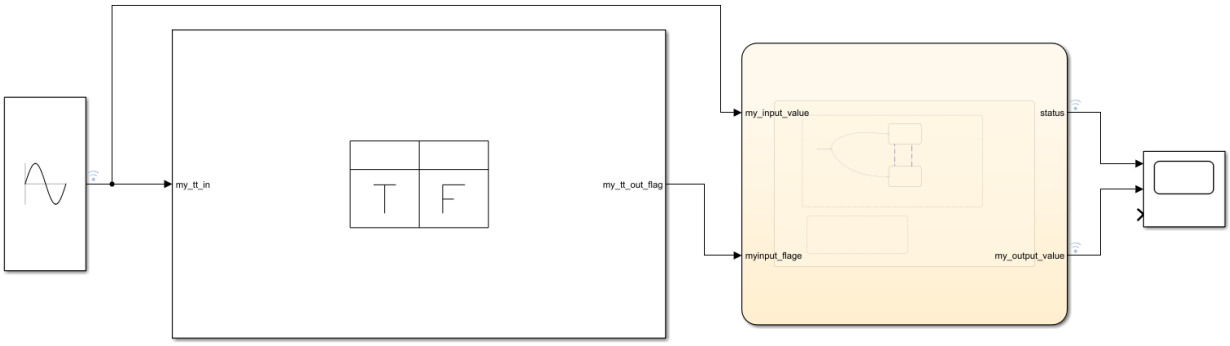

Figure 12.superstate_model

my_output_value = 2 * my_input_value; was removed and replaced with a Simulink Function block. In the new setup, the State flow chart sends the input value to the Simulink Function. Inside the function block, a simple MATLAB function multiplies the input by two and returns the result. The output from the function is then passed back into the chart.

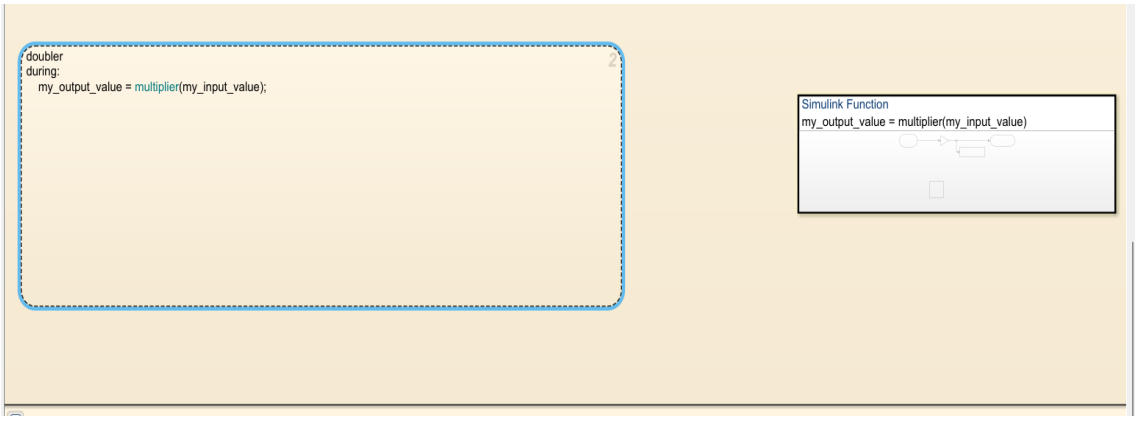

Figure 14Modified State flow Logic Calling a Simulink Function

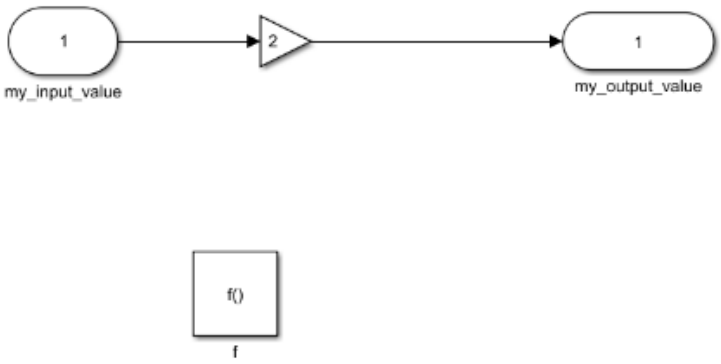

Figure 13. Simulink Function for Signal Multiplication

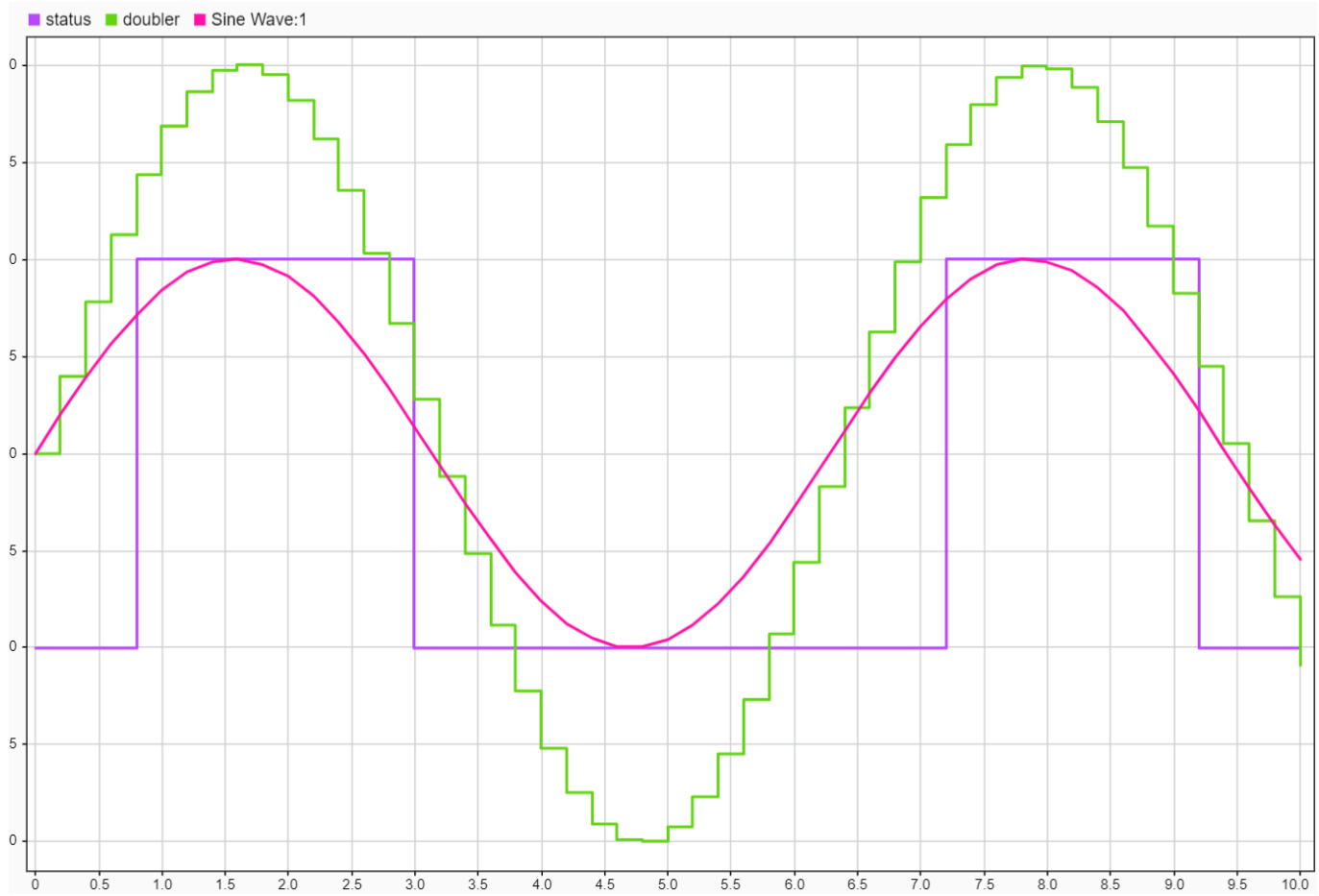

Figure 15. Status and Doubled Signal Compared to Input Sine Wave

The sine wave is fed into the superstate. When it crosses the threshold, the purple line flips between 0 and 1. At the same time, the green curve shows the input being doubled by the Simulink Function.

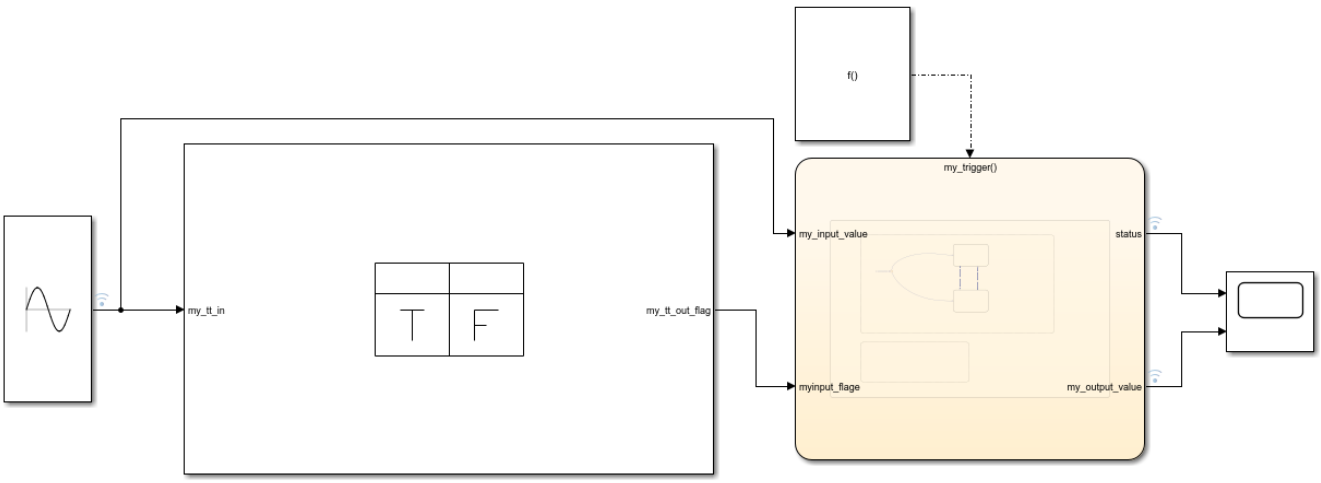

Figure 16.Superstate with External Function Trigger

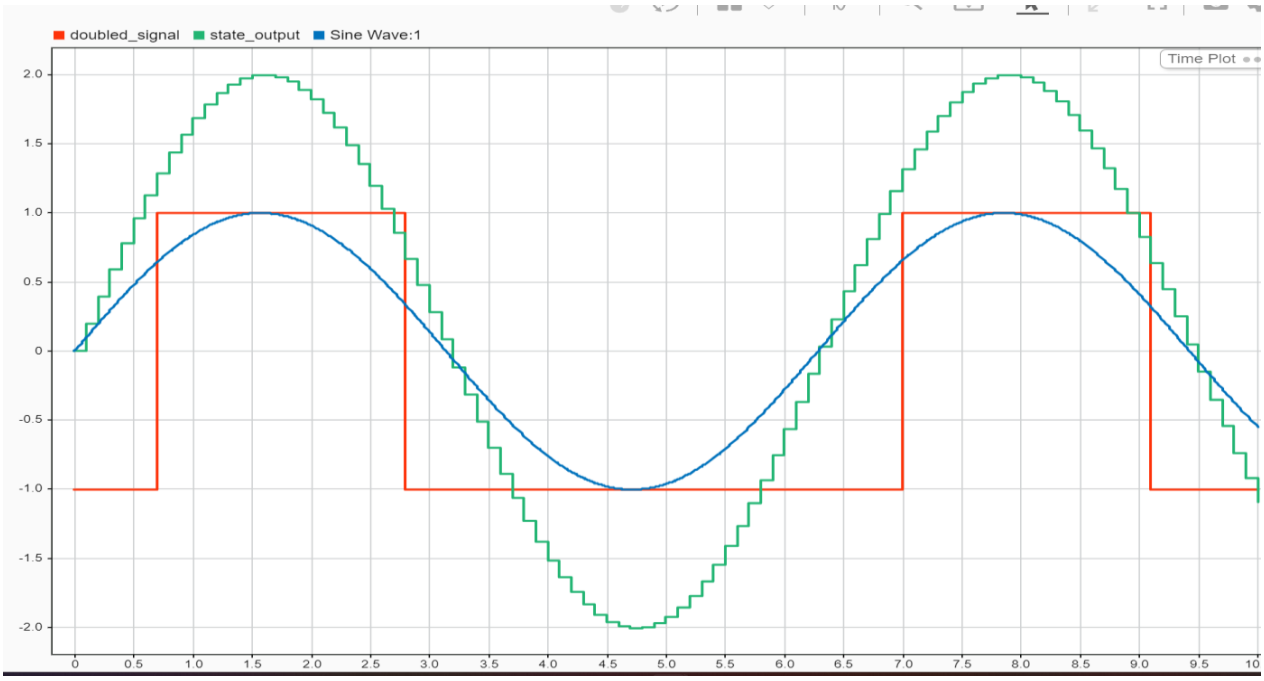

Figure 17. Simulation with Triggered State flow Execution

After adding the trigger function, the chart updates only when a trigger event occurs, not continuously. Because of this, the status and doubled signals change in discrete steps. The steps appear smaller compared to the version without a trigger because the trigger runs at a lower, fixed rate, so the chart receives fewer updates per second

6.Code export

6.1 LED Blink and Sensor Test in Arduino IDE

Example code from the Arduino IDE that toggles the built-in LED on and off every second. Uploading and running this sketch verifies that the MKR1000/1010 board is functioning correctly and able to execute user code.

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);  // turn the LED on (HIGH is the voltage level)
  delay(1000);                      // wait for a second
  digitalWrite(LED_BUILTIN, LOW);   // turn the LED off by making the voltage LOW
  delay(1000);                      // wait for a second
}
```

Figure 18 Arduino "Blink" Example Used to Test the

```
// digital pin 2 has a pushbutton attached to it. Give it a name:
int pushButton = 2;

// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
  // make the pushbutton's pin an input:
  pinMode(pushButton, INPUT);
}

// the loop routine runs over and over again forever:
void loop() {
  // read the input pin:
  int buttonState = digitalRead(pushButton);
  // print out the state of the button:
  Serial.println(buttonState);
  delay(1);  // delay in between reads for stability
}
```

Figure 19.DigitalReadSerial Example Used to Test Sensor Input

6.2 Exporting Code from Simulink and Reading Sensor Data

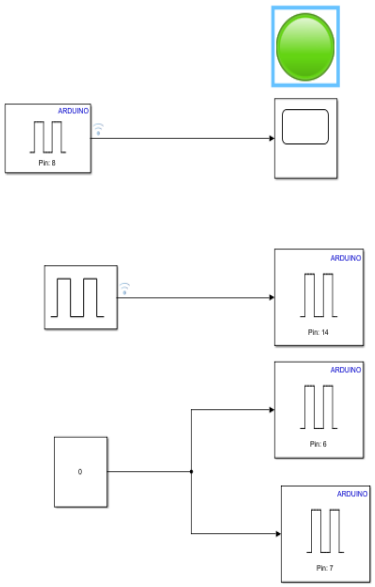

Figure 20. Simulink Model for Reading Line Sensor Data

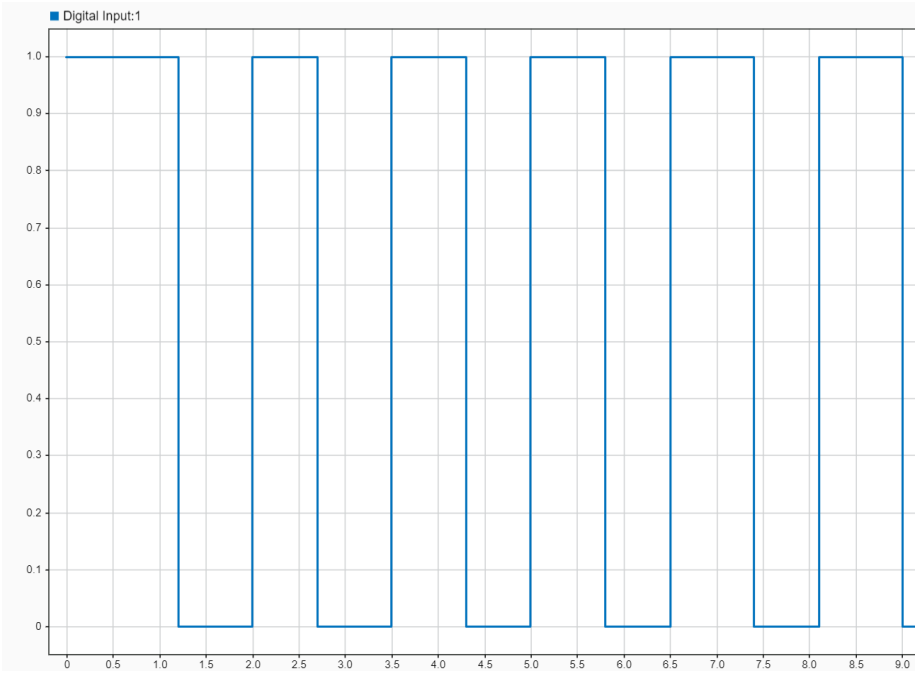

Figure 21. Digital Input Signal Read from the Line Sensor

The live sensor output from the Arduino MKR board displayed in a Simulink Scope. The digital line sensor alternates between 1 and 0, confirming that the hardware is powered, the input pins are working, and External Mode communication is functioning correctly.

6.3 Reading IMU Data Using the Mecroka Library

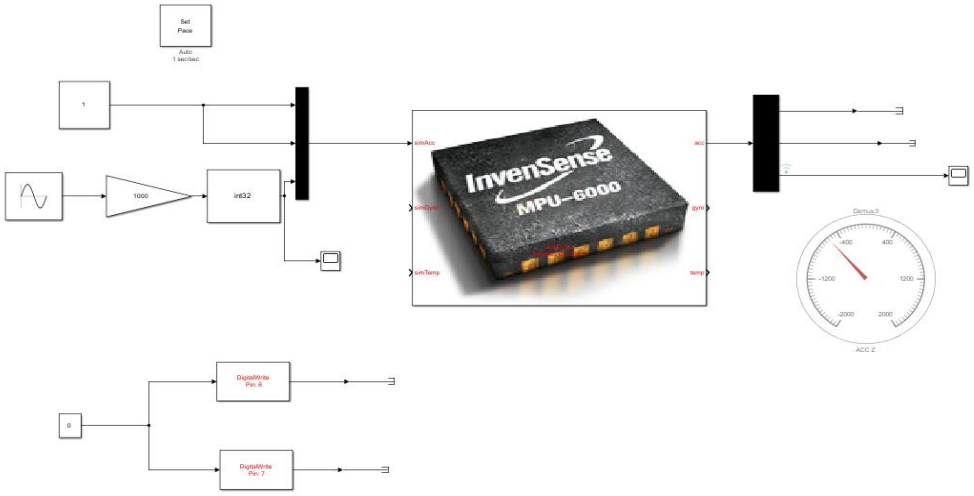

Figure 20.Simulink Model for IMU Acceleration

This model is used to simulate and read accelerometer data from the MPU-6000 IMU using the Mecroka library. After scaling, the signal is converted to int32 and supplied to the Simac input port of the IMU block as a three-axis acceleration vector.

The IMU block outputs acceleration values through the acc port, which are visualised using both a numeric display and a gauge. A “Set Pace” block is included to slow the simulation down to real-time speed, allowing the changes in acceleration to be observed smoothly during simulation.

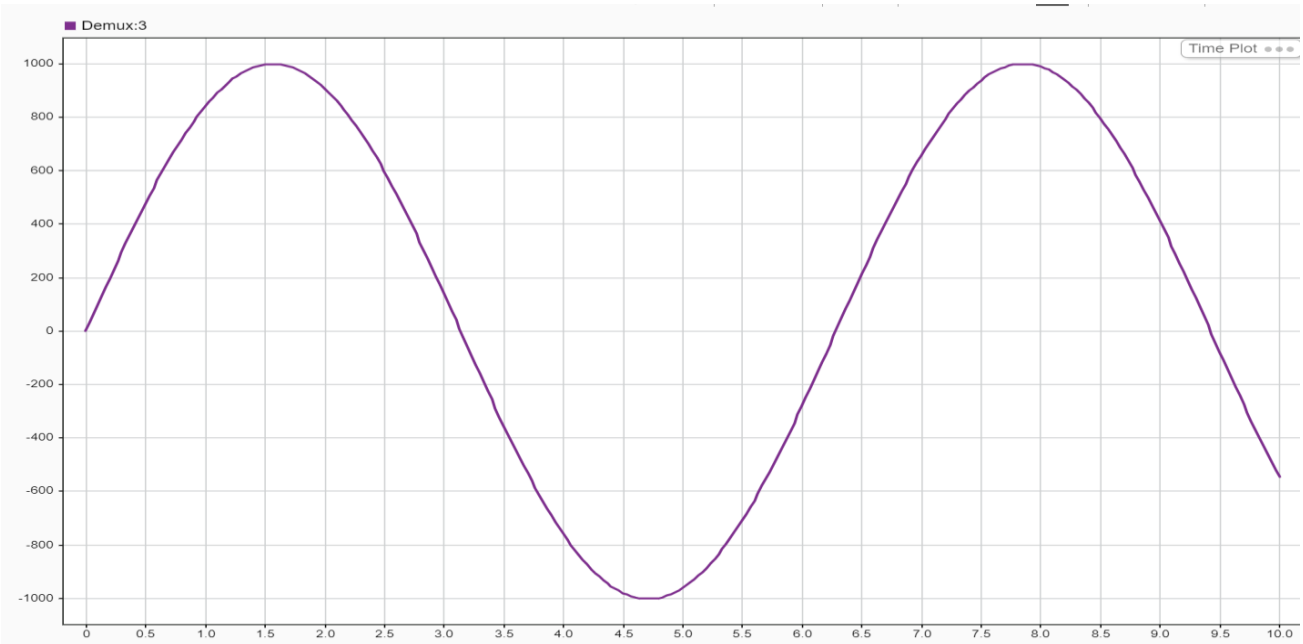

Figure 23. Simulated IMU Acceleration Signal in Virtual Environment

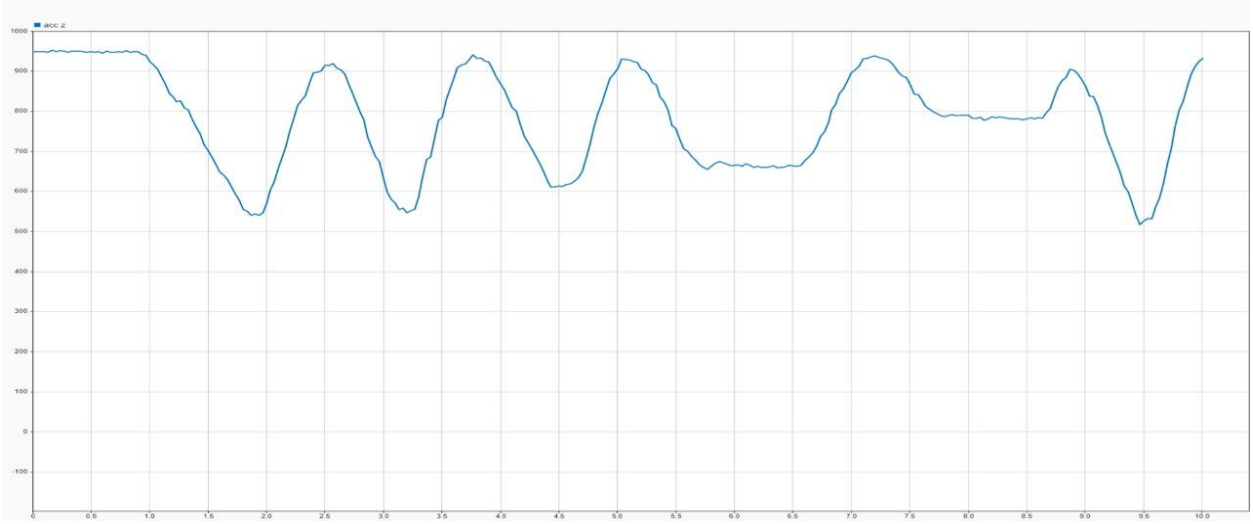

Figure 24. Real IMU Acceleration Signal Measured on the Robot

Video: -

[Real_time_simulation_video]

7. Setting up and testing the two-wheel robot

When the simulation starts, the virtual robot begins moving and turning based on the default controller, giving an initial view of how the two-wheel robot behaves in the simulated environment.

Video: -

virtual_simulation of_two_wheel_robot

After completing the simulation, the controller model is exported from Simulink and uploaded to the robot's hardware board. Once the controller is running on the actual robot, its behaviour can be observed directly. The robot responds to the control signals in real time, performing turns and forward motion similar to the simulated model. The video below demonstrates how the robot behaves on hardware with the deployed controller.

Video: -

Real_time_simulation